## What causes a deadlock?

A deadlock situation occurs when each thread waits for a resource that's being held by another thread and hence neither thread can make any progress. In last month's takeaway problem, we saw that each thread waited for the lock held by another thread, and hence couldn't make any progress.

Deadlock is one of the most common and highly dreaded bugs in multithreaded code. In this month's column, we will discuss what conditions can cause deadlock to occur, the techniques for deadlock prevention, etc.

There are two types of deadlocks—resource deadlocks and communication deadlocks. In a resource deadlock, threads (or processes, as in the case of a multi-process application as opposed to a multi-threaded application) are in a circular queue, waiting for resources currently owned by another thread, which in turn waits for a resource owned by this thread. A set of threads (processes) is resource deadlocked if each thread in the set requests a resource held by another thread (process) in the set.

In communication deadlocks, messages are the resources for which threads wait. A set of threads (processes) is communication deadlocked if each thread (process) in the set is waiting for a message from another thread (process) in the set, and no thread (process) in the set ever sends a message. Communication deadlocks are important in the world of message passing programming, where distributed processes communicate using messages. Over the rest of this column, we will focus our attention on resource-based deadlocks.

## Conditions leading to deadlock

So what are the conditions that can lead to a deadlock situation? As we have seen in the earlier example, a circular wait among the threads is one of the conditions that can lead to deadlock. There are four conditions that must simultaneously occur, for a deadlock to happen. They are:

1. **The mutual exclusion condition:** The resources that are being contended for by the threads are not shareable. For example, consider our example with the critical sections, 'BookTicket' and 'CancelTicket'. In these sections, the rows and columns of the reservation tables are not shareable. Hence, the programmer has protected access to these resources using mutex locks. So our example deadlock satisfies the first condition.
2. **The 'resource hold and wait' condition**: There is a set of threads in which each thread holds a resource already allocated to it while waiting for additional resources that are currently being held by other threads. In our example, we have Thread 1 in the 'BookTicket' critical section, which holds the 'row_lock' and is requesting for the 'column_lock' critical section. We have Thread 2 in the 'CancelTicket' critical section, holding the 'column_lock' and requesting for 'row_lock'. Hence our example satisfies the condition of 'resource hold and wait'.
3. **The 'no pre-emption' condition**: Resources already allocated to a thread cannot be pre-empted and given to another thread. For instance, in our example code, the operating system cannot pre-empt Thread 2 to relinquish the 'column_lock' to Thread 1. So the 'no pre-emption' condition holds for our example deadlock.
4. **Circular wait condition:** The threads in the set form a circular list or chain where each thread in the list is waiting for a resource held by the next thread in the list. In our example, we have Thread 1 and Thread 2 as the elements of the circular list, with Thread 1 waiting for 'column_lock' and Thread 2 waiting for 'row_lock'. So our example satisfies the fourth condition needed for a deadlock to occur.

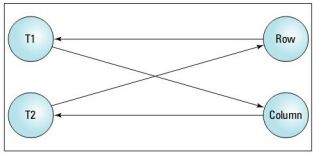


Figure 1: Example of a resource allocation graph

These four conditions must be satisfied simultaneously for a program to reach a deadlock state, after which, it remains forever in that state since no thread can make progress. Therefore the program needs to be killed and restarted by the programmer.

In order to understand the complex interactions between resources available and the threads that use these resources, a resource allocation graph is used to represent the interaction. The resource allocation graph is a bipartite directed graph, wherein resources and threads are the nodes of the graph. One partition consists of resource nodes and another partition consists of thread nodes. All the edges of the graph go between the two partitions. There are no edges between the nodes of the same partition. An edge exists from a resource to the thread to which it is allocated. Such an edge is